

Design Patterns Refactoring – Parkway Project

Overview

- **Branch:** feature/design-patterns-refactor
- **Commits:** 2 (Backend + Frontend)
- **Lines Added:** 900+ | **Duplicate Code Removed:** ~400 lines

Patterns Applied

# Pattern	Location	Purpose	Status
1 Adapter + Facade	web/src/services/apiClient.js	Centralize API calls, remove hardcoded URLs	 Complete
2 Factory	backend/.../util/DTOMapper.java	Consolidate duplicate DTO conversions	 Complete
3 @ControllerAdvice	backend/.../exception/GlobalExceptionHandler.java	Remove repeated try-catch blocks	 Complete
4 Facade	backend/.../service/AdminCreationFacade.java	Single method for admin + slot creation	 Complete
5 Custom Hooks	web/src/hooks/*.js	Simplify scattered	 Complete

# Pattern	Location	Purpose	Status
		useState logic	

Key Improvements

1. Frontend – API Client Service

Problem: Components used 12+ hardcoded API endpoints, inconsistent error handling.

Solution: apiClient.js centralizes all API calls and supports environment switching (.env.local).

Before:

```
fetch('http://localhost:8080/api/users/login', {...})
fetch(`http://localhost:8080/api/bookings/user/${userId}`)
```

After:

```
import { authAPI, bookingAPI } from './services/apiClient';
const user = await authAPI.login(email, password);
const bookings = await bookingAPI.getUserBookings(userId);
```

Benefits:

- Single source of truth for URLs
 - Easy dev/prod switching
 - Consistent error handling
-

2. Backend – DTO Factory

Problem: Duplicate conversion logic in 5+ services.

Solution: DTOMapper centralizes all entity → DTO conversions.

Before: Each service manually mapped fields.

After:

@Autowired

private DTOMapper dtoMapper;

```
List<BookingDTO> bookings = bookingRepository.findByUser(userId)
    .stream()
    .map(dtoMapper::toBookingDTO)
    .collect(Collectors.toList());
```

Benefits:

- Removes ~50 lines of duplicate code
 - Null-safe, reusable, testable
 - Easier maintenance
-

3. Backend – Global Exception Handler

Problem: 50+ repeated try-catch blocks with inconsistent responses.

Solution: @RestControllerAdvice handles exceptions globally.

Before: Each controller manually caught exceptions.

After: Controllers are clean; exceptions handled in one place.

Benefits:

- Consistent error responses
 - Centralized logging
 - Simplifies controller code
-

4. Backend – Admin Creation Facade

Problem: Admin registration repeated slot creation logic in 3 places.

Solution: Facade provides one atomic method to register admin + slots.

Benefits:

- Eliminates duplicate code
- Atomic operation with @Transactional

- Clear, maintainable code
-

5. Frontend – Custom Hooks

Problem: 12–15 useState calls scattered across components.

Solution: Create reusable hooks: useModalState, useFormHandler, useCurrentUser.

Benefits:

- Reduces 40+ useState calls to a few hooks
 - Reusable, testable
 - Cleaner component code
-

Integration Checklist

Backend:

- Inject DTOMapper where needed
- Replace manual conversions with dtoMapper.to*DTO()
- Remove try-catch in controllers (rely on GlobalExceptionHandler)
- Use AdminCreationFacade for admin registration

Frontend:

- Create .env.local
- Replace direct fetch calls with apiClient methods
- Replace scattered useState with custom hooks
- Test API calls and environment switching

Testing:

- Backend: mvn test
- Frontend: npm test
- Manual: simulate API errors, verify state handling